

Aula 12 - IPsec

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
October 24, 2025

IPsec: O Protocolo IP de Segurança

- Provê segurança na camada de rede.
- Protege datagramas IP entre quaisquer entidades (dispositivos finais (*hosts*), roteadores).
- Usado para criar Redes Privadas Virtuais (VPNs) sobre a Internet pública.
- Definido pelo IAB como essencial para o IPv6 (autenticação e criptografia).
- Projetado para ser compatível tanto com IPv4 quanto com IPv6.
- Atualmente, muitos fabricantes já oferecem suporte ao IPsec.

Relembrando encapsulamento em redes

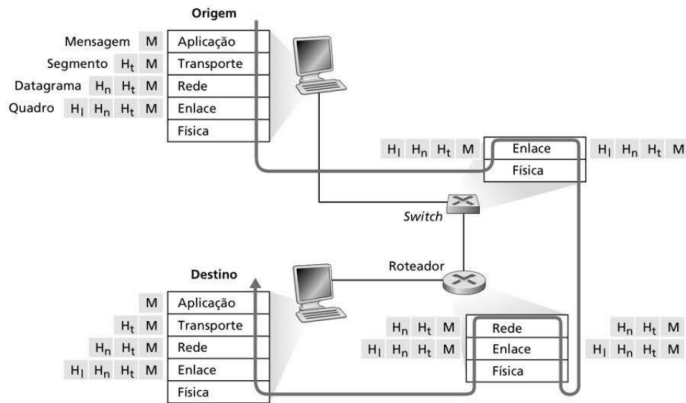


Figura 1.24 Hospedeiros, roteadores e switches; cada um contém um conjunto diferente de camadas, refletindo suas diferenças em funcionalidade.

Relembrando o formato do datagrama IPv4

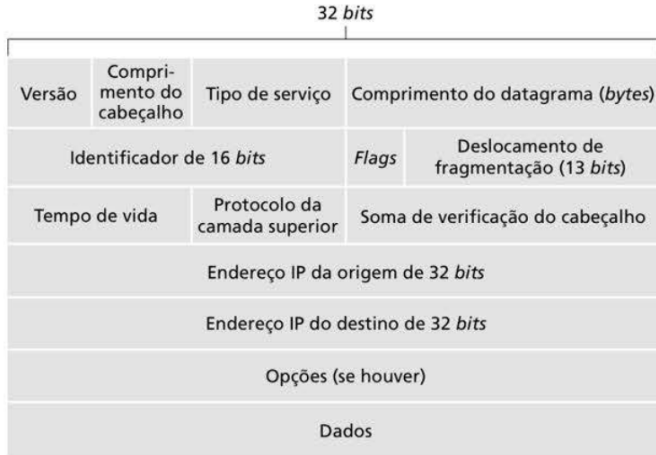


Figura 4.17 Formato do datagrama IPv4.

Áreas Funcionais da Segurança em Nível IP

- A segurança em nível IP abrange três áreas funcionais:

1. Autenticação

- Garante que o pacote foi realmente enviado pela fonte identificada.
- Assegura que o pacote não foi alterado durante o trânsito.

2. Confidencialidade

- Permite aos nós da rede cifrar (criptografar) mensagens.
- Previne a espionagem (eavesdropping) por terceiros.

3. Gerenciamento de Chaves

- Trata da troca segura de chaves entre as partes.

O Conceito de Sigilo na Camada de Rede

- O que significa prover sigilo na camada de rede?
 - A entidade remetente (hospedeiro ou roteador) cifra as **cargas úteis (payloads)** de todos os datagramas que envia.
- O que é a carga útil (payload)?
 - Segmento TCP
 - Segmento UDP
 - Mensagem ICMP
 - Mensagem SNMP, etc.
- Resultado: "**Cobertura Total**"
 - Todos os dados (e-mails, páginas Web, mensagens de gerenciamento) ficam ocultos de intrusos.

Além da confidencialidade, um protocolo de segurança da camada de rede pode prover:

- **Autenticação da Origem**
 - Permite à entidade destinatária verificar a origem (remetente) do datagrama protegido.
- **Integridade dos Dados**
 - Permite à entidade destinatária verificar se o datagrama sofreu qualquer adulteração durante o trânsito.
- **Prevenção do Ataque de Repetição (Replay Attack)**
 - Permite ao destinatário (Bob) detectar quaisquer datagramas duplicados que um atacante possa inserir no fluxo.
- **Característica:** IPsec pode **criptografar e/ou autenticar todo o tráfego no nível IP.**

Aplicações Comuns do IPsec

- O IPsec pode ser usado em LANs, WANs (públicas/privadas) e na Internet.
- Todas as aplicações distribuídas se beneficiam, sem necessidade de modificação.
- **Exemplos de uso:**
 - **Conectividade Segura de Filiais (VPN):**
 - Permite construir uma VPN segura sobre a Internet.
 - Reduz custos e a necessidade de redes privadas.
 - **Acesso Remoto Seguro:**
 - Usuários remotos (viajantes, telecommuters) acessam a rede da empresa de forma segura através de um ISP local.
 - **Conectividade Extranet/Intranet com Parceiros:**
 - Protege a comunicação com outras organizações (autenticação, confidencialidade, troca de chaves).
 - **Reforço à Segurança de E-commerce:**
 - Adiciona segurança, mesmo que as aplicações Web já possuam protocolos de segurança próprios.

Benefícios do IPsec

- **Implementação em Firewall/Roteador**
 - Aplica segurança forte a todo o tráfego que cruza o perímetro.
 - O tráfego interno (workgroup) não sofre overhead de processamento.
- **Resistência a Bypass**
 - Difícil de contornar (bypass) se o firewall IPsec for a única entrada da Internet.
- **Transparência para Aplicações**
 - Atua abaixo da camada de transporte (TCP/UDP).
 - Não exige mudança de software em usuários ou servidores.
 - Aplicações de camada superior não são afetadas.
- **Transparência para o Usuário Final**
 - Não é necessário treinar os usuários sobre os mecanismos de segurança.
 - Não é preciso gerenciar chaves por usuário (emitir/revogar).
- **Flexibilidade (Segurança Individual)**
 - Pode prover segurança para usuários individuais, se necessário.
 - Útil para trabalhadores remotos (off-site).
 - Permite criar sub-redes virtuais seguras para aplicações sensíveis.

IPsec e Redes Privadas Virtuais (VPNs)

- **O Cenário:**

- Instituições que se estendem por diversas regiões geográficas.

- **A Necessidade:**

- Desejam ter sua própria rede IP.
- Objetivo: Permitir que seus hóspedes e servidores enviem dados uns aos outros de forma segura e sigilosa.

- **A Solução Tradicional: Rede Privada**

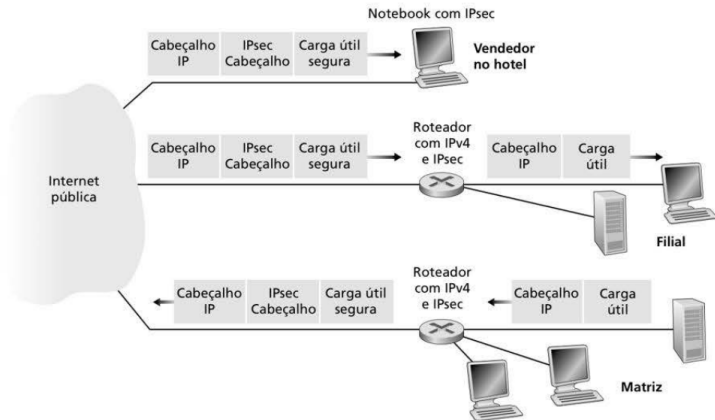
- Definição: Uma rede física independente, reservada a uma instituição particular.
- Características:
 - É completamente separada da Internet pública.
 - Requer infraestrutura própria: roteadores, enlaces e DNS.

- **O Problema da Rede Privada:**

- Custo muito elevado.
- A instituição precisa comprar, instalar e manter sua própria infraestrutura de rede física.

A Solução: Rede Privada Virtual (VPN)

- VPNs operam "em cima" (sobre) da Internet pública.
- O tráfego é **criptografado** e enviado através da infraestrutura da Internet pública.
- Não é necessária uma rede fisicamente independente e dedicada.
- A criptografia é aplicada **antes** que os pacotes entrem na Internet pública.



VPN: Cenário de Exemplo e Fluxo de Tráfego

- **Componentes da Instituição (Exemplo):**
 - Matriz, Filial e outros acessando da Internet
- **Fluxo 1: Comunicação Interna (Dentro de um Local)**
 - Ex: Hospedeiro na Matriz ↔ Hospedeiro na Matriz.
 - Ex: Hospedeiro na Filial ↔ Hospedeiro na Filial.
 - Método: IP v4 padrão (sem IPsec). O tráfego não sai para a rede pública.
- **Fluxo 2: Comunicação Externa (Entre Locais via VPN)**
 - Ex: Hospedeiro na Matriz ↔ Hospedeiro na Filial.
 - Ex: Vendedor Viajante ↔ Hospedeiro na Matriz.
 - Caminho: O tráfego deve cruzar a Internet pública.
 - Método: O tráfego é **codificado (criptografado)** antes de entrar na Internet (usando IPsec).

Tráfego Misto: IPsec e IPv4 Comum

- **Ponto Importante:**

- Nem todo o tráfego enviado à Internet pelos roteadores de borda ou pelos notebooks será protegido por IPsec.

- **Exemplo: Acesso à Internet Pública**

- Um hospedeiro na matriz quer acessar um servidor Web comum (ex: Amazon, Google).
- Este tráfego não é destinado à VPN.

- **Resultado: Coexistência de Tráfego**

- O roteador de borda (e os notebooks) emitirão **ambos** os tipos de datagramas para a Internet:
 - Datagramas IPv4 comuns (para tráfego público).
 - Datagramas IPsec seguros (para tráfego da VPN).

VPN: Fluxo de Tráfego (Matriz → Vendedor Viajante)

- **1. Na Matriz (Origem)**
 - Um hospedeiro envia um datagrama IPv4 padrão (ex: TCP/UDP).
- **2. No Roteador de Borda (Gateway IPsec da Matriz)**
 - Intercepta o datagrama IPv4.
 - Converte o datagrama IPv4 em um datagrama **IPsec**.
 - Encaminha o pacote IPsec para a Internet.
- **3. Na Internet (Trânsito)**
 - O datagrama IPsec possui um cabeçalho IPv4 tradicional (externo).
 - Roteadores da Internet o processam como um datagrama IPv4 comum.
- **4. Estrutura do Pacote IPsec (no Trânsito)**
 - A carga útil (payload) do datagrama IPsec contém:
 - Um **cabeçalho IPsec** (para processamento de segurança).
 - A carga útil original (ex: segmento TCP/UDP), que está **codificada** (criptografada).
- **5. No Notebook do Vendedor (Destino)**
 - O S.O. recebe o datagrama IPsec.
 - **Decodifica** a carga útil (descriptografa).
 - Verifica outros serviços (ex: integridade dos dados).
 - Passa a carga útil original (não codificada) para a camada superior (TCP ou UDP).

Protocolos Principais do IPsec: AH vs. ESP

O conjunto IPsec possui dois protocolos principais para envio de dados seguros (entre hospedeiros ou roteadores):

- **1. Protocolo Cabeçalho de Autenticação (AH - Authentication Header)**
 - **Provê** Autenticação da Origem.
 - **Provê** Integridade dos Dados.
 - **NÃO** provê Sigilo (Confidencialidade).
- **2. Protocolo Carga de Segurança de Encapsulamento (ESP - Encapsulating Security Payload)**
 - **Provê** Autenticação da Origem.
 - **Provê** Integridade dos Dados.
 - **Provê Sigilo (Confidencialidade)** ← (Função combinada de autenticação/criptografia).

O Foco no ESP (AH é Deprecated)

- **Por que o ESP é muito mais utilizado?**
 - O Sigilo (Criptografia) é muitas vezes essencial para VPNs.
 - VPNs geralmente desejam **autenticação E criptografia** para:
 - 1. Impedir que usuários não autorizados penetrem na rede.
 - 2. Impedir que espiões (eavesdroppers) na Internet leiam as mensagens.
- **O Status Atual do AH: é deprecated (obsoleto)**
 - Motivo: O próprio protocolo ESP já fornece a autenticação da mensagem.
 - O AH é mantido no IPsecv3 apenas para *backward compatibility* (compatibilidade retroativa).
 - **Não deve ser usado em novas aplicações.**
- **Foco atual: ESP.**

Associações de Segurança (SA)

- Datagramas IPsec são enviados entre pares de entidades da rede (hospedeiro ↔ hospedeiro, roteador ↔ roteador, etc.).
- **O que é uma Associação de Segurança (SA - Security Association)?**
 - É uma **conexão lógica** da camada de rede.
 - Criada **antes** que a entidade remetente possa enviar datagramas IPsec à entidade destinatária.
- **Característica Chave: Unidirecional**
 - Uma SA é uma conexão lógica **simples** (simplex).
 - Ela flui em apenas uma direção: do remetente para o destinatário.
- **Comunicação Bidirecional (Mútua)**
 - Se duas entidades querem enviar dados seguros entre si (nos dois sentidos): Precisam ser estabelecidas **duas SAs** (duas conexões lógicas).
 - Uma SA em cada direção.

Exemplo: Contagem de SAs em uma VPN

- **Cenário (Instituição):**
 - 1 Matriz.
 - 1 Filial.
 - n Vendedores Viajantes (notebooks).
- **Fluxos de Tráfego IPsec (Bidirecional):**
 - Tráfego bidirecional entre o roteador de borda da Matriz e o roteador de borda da Filial.
 - Tráfego bidirecional entre o roteador de borda da Matriz e CADA UM dos n Vendedores.
- **Cálculo - Quantidade de SA:**
 - **Matriz $\leftrightarrow$ Filial:**
 - 1 SA (Matriz $\rightarrow$ Filial)
 - 1 SA (Filial $\rightarrow$ Matriz)
 - *Subtotal* = 2 SAs
 - **Matriz $\leftrightarrow$ n Vendedores:**
 - Para cada vendedor, são necessárias 2 SAs (uma em cada direção).
 - *Subtotal* = $2 \times n$ SAs
- **Total de SAs na VPN:**
 - Total = (SAs da Filial) + (SAs dos Vendedores)
 - Total = $2 + 2n$ SAs

O Banco de Dados de Associações de Segurança (SAD)

- Toda implementação IPsec inclui um **Banco de Dados de Associações de Segurança** (SAD - Security Association Database).
- Este banco de dados armazena e define todos os parâmetros associados a cada SA (Associação de Segurança) ativa.
- Uma SA é caracterizada por um conjunto de parâmetros específicos. **Que vamos ver nos próximos slides**
- Uma entidade IPsec (roteador ou hospedeiro) armazena informações de estado para **muitas SAs** simultaneamente.
- **Relembrando o Exemplo da VPN:**
 - Cenário: 1 Matriz, 1 Filial e n vendedores.
 - O roteador de borda da Matriz guarda informações de estado para $(2 + 2n)$ SAs.

O SPD - Security Policy Database

O Problema: R1 recebe um datagrama IPv4 da rede interna destinado a um IP de fora.

- **Como R1 sabe** se este datagrama deve ser convertido em IPsec (ou enviado como IPv4 comum)?
 - ... e se for IPsec, qual SA (dentre as várias no SAD) ele deve usar?
- **A Solução: O Banco de Dados de Política de Segurança (SPD)**
 - A entidade IPsec (R1) mantém esta *outra* estrutura de dados (além do SAD).
 - O SPD indica **quais tipos** de datagramas serão processados pelo IPsec.
 - A decisão é baseada em: Endereço IP remetente, Endereço IP destinatário, e Tipo do protocolo (TCP/UDP).
 - O SPD também aponta para qual SA deve ser usada.
- **A Diferença Chave: SPD vs. SAD**
 - **SPD (Política):** Indica "**O QUE**" fazer com um datagrama (Processar com IPsec, Descartar, Deixar passar).
 - **SAD (Associação):** Indica "**COMO**" fazer o IPsec (Chaves de criptografia/autenticação, algoritmos, SPI, etc).

Informações de Estado da SA (Exemplo: Roteador R1)

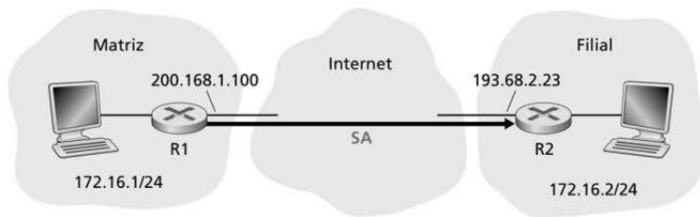


Figura 8.28 Associação de segurança (SA) de R1 a R2.

Uso das Informações de Estado da SA (Exemplo $R1 \rightarrow R2$)

- **No Remetente (Roteador R1)**

- Quando R1 precisa construir um datagrama IPsec para encaminhar por essa SA:
- Ele acessa as informações de estado (SPI, chaves, algoritmos) da SA.
- Usa essas informações para determinar como deve **autenticar** e **criptografar** o datagrama.

- **No Destinatário (Roteador R2)**

- R2 mantém as *mesmas* informações de estado sobre essa SA (identificada pelo SPI).
- Ele as usa para **autenticar** (verificar a integridade) e **decodificar** (descriptografar) qualquer datagrama IPsec que chegar da SA.

Informações de Estado da SA (Exemplo: Roteador R1)

O Roteador R1 manterá as informações de estado sobre essa SA:

- Um identificador de 32 bits para a SA: **Índice de Parâmetro de Segurança (SPI)**
- As interfaces da SA:
 - Interface remetente (Origem): 200.168.1.100
 - Interface destinatária (Destino): 193.68.2.23
- Parâmetros de Criptografia (para Sigilo):
 - O tipo de criptografia (ex: 3DES com CBC).
 - A chave de criptografia.
- Parâmetros de Integridade (para Autenticação):
 - O tipo de verificação de integridade (ex: HMAC com MD5).
 - A chave de autenticação.

Parâmetros da SA (Parte 1)

- **Contador de Número de Sequência (Sequence number counter)**
 - Um valor de 32 bits usado para gerar o campo "Sequence Number" nos cabeçalhos AH ou ESP
 - Essencial para o antireplay.
- **Estouro do Contador (Sequence counter overflow)**
 - Uma "flag" que indica se o estouro (overflow) do contador deve:
 - Gerar um evento auditável (log).
 - Impedir a transmissão de mais pacotes nesta SA.
- **Janela Antireplay (Antireplay window)**
 - Define uma "janela deslizante" (sliding window).
 - Usada para determinar se um pacote AH ou ESP recebido é um *replay* (pacote duplicado/malicioso).
 - O número de sequência do pacote deve estar dentro desta janela.

Parâmetros da SA (Parte 2)

- **Informações ESP**

- Algoritmo de criptografia e autenticação.
- Chaves, Valores de Inicialização (IVs), tempos de vida das chaves e parâmetros relacionados ao ESP.

- **Tempo de Vida da SA (Lifetime)**

- Um intervalo de tempo ou uma contagem de bytes.
- Após atingido, a SA deve ser substituída por uma nova SA (e novo SPI) ou terminada.

- **Modo do Protocolo IPsec (Protocol mode)**

- Define o modo de operação: Túnel (Tunnel) ou Transporte (Transport)

- **Path MTU**

- Armazena o "Maximum Transmission Unit" (tamanho máximo do pacote) observado no caminho.
- Usado para evitar fragmentação de pacotes.
- Inclui variáveis de "envelhecimento" (aging).

O Datagrama IPsec: Modo Túnel vs. Modo Transporte

- Analisando o datagrama IPsec.
- O IPsec possui **duas formas diferentes de pacotes** (dois modos de operação):
 - **Modo Túnel (Tunnel Mode)**
 - **Modo Transporte (Transport Mode)**
- **Comparação de Uso:**
 - O **Modo Túnel** é o mais apropriado para o cenário de VPNs (que vimos anteriormente).
 - Por esta razão, o Modo Túnel é **mais implementado** do que o Modo Transporte.
- **Foco da Aula:**
 - A partir de agora vamos nos concentrar **exclusivamente no Modo Túnel**.
 - (Uma vez que se tem uma compreensão sólida do Modo Túnel, é fácil aprender o Modo Transporte).

Processo de Criação do Datagrama ESP (Modo Túnel) - Parte 1

O processo de encapsulamento (ex: no roteador de borda) segue estes passos

- **1. Criptografia e criação da mensagem**

- Pega-se o datagrama IP original (que será protegido).
- Criptografa-se este datagrama (ele se torna o campo "Payload Data").
- Anexa-se na frente dele o "Cabeçalho ESP" (SPI + Seq. Number).
- O pacote resultante é o Cabeçalho ESP + Carga Criptografada + Trailers ESP.

- **2. Autenticação (MAC/ICV)**

- Cria-se uma autenticação MAC (ou ICV - Integrity Check Value) sobre o datagrama inteiro.
- Utiliza-se o algoritmo (ex: HMAC) e a chave de autenticação especificados na SA.

- **3. Formação da Carga Útil (Payload) ESP**
 - Anexa-se o MAC (ICV) atrás do datagrama.
 - O resultado (Cabeçalho ESP + Carga Criptografada + Trailers + MAC) forma a **Carga Útil ESP** completa.
- **4. Criação do Novo Cabeçalho IP (Modo Túnel)**
 - Por fim, cria-se um **novo cabeçalho IP** (IPv4 clássico, 20 bytes).
 - Este novo cabeçalho é anexado **antes** da Carga Útil ESP.
 - Este é o cabeçalho que os roteadores da Internet irão ler.

Datagrama IPsec

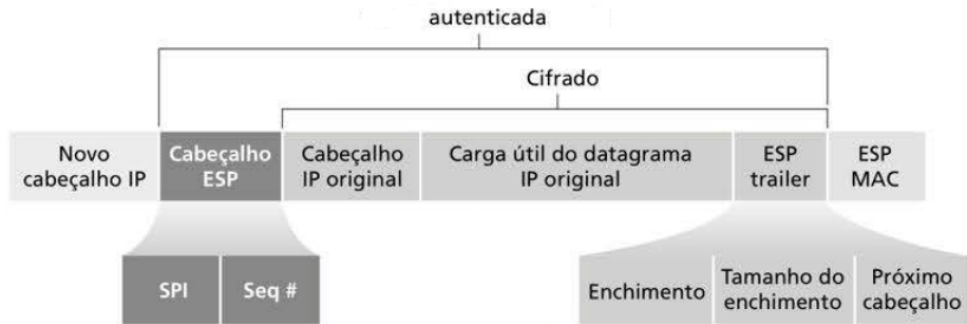


Figura 8.29 Formato do datagrama IPsec.

O Datagrama IPsec Resultante (Modo Túnel)

- **A Carga Útil (Payload) deste datagrama contém:**
 - O Cabeçalho ESP (SPI, Seq. Number).
 - O **Datagrama IP Original** (cifrado).
 - O Trailer ESP (Padding, Pad Length, Next Header) (cifrado).
 - O Campo de Autenticação ESP (ICV/MAC).
- **Cabeçalho IP Original (Interno e Cifrado)**
 - Contém os endereços dos *hospedeiros finais*.
 - Endereço IP Remetente (Ex): 172.16.1.17
 - Endereço IP Destinatário (Ex): 172.16.2.48
 - Estes endereços são **cifrados** e invisíveis para a Internet.
- **Novo Cabeçalho IP (Externo e Visível)**
 - Contém os endereços das *extremidades do túnel* (os gateways IPsec).
 - Endereço IP Remetente (Ex): 200.168.1.100
 - Endereço IP Destinatário (Ex): 193.68.2.23
 - O campo "Protocolo" deste novo cabeçalho é **50** (indicando ESP), e não TCP(6) ou UDP(17).

Revisão: O Cabeçalho ESP (Enviado em Aberto)

- O "Cabeçalho ESP" é anexado na frente da unidade cifrada (payload + trailer).
- Este cabeçalho **não é criptografado** (é enviado "em aberto" / "clear text").
- Ele consiste em dois campos principais:
 - **1. O Índice de Parâmetro de Segurança (SPI)**
 - **Função:** Indica à entidade destinatária (ex: R2) a qual Associação de Segurança (SA) este datagrama pertence.
 - **Ação no Destino:** A entidade usa o SPI para indexar (localizar) rapidamente a SA correta em seu Banco de Dados (SAD).
 - Ao localizar a SA, o destino sabe quais algoritmos e chaves deve usar para autenticar e decriptar o pacote.
 - **2. O Campo de Número de Sequência**
 - **Função:** É usado para a proteção contra ataques de repetição (replay attacks).
 - (O destinatário verifica este número contra a sua "janela antireplay" definida na SA).

O Trailer ESP: Campos e Criptografia

- O trailer ESP consiste em três campos:
- **Importante:** O trailer é adicionado ao datagrama IP original **antes** da cifragem.
- **1. Enchimento (Padding)**
 - **O quê:** Bytes sem significado.
 - **Por quê:** Cifras de bloco exigem que a mensagem a ser cifrada seja um múltiplo inteiro do comprimento de bloco (ex: 128 bits para o AES).
 - O enchimento é usado para "completar" o bloco.
- **2. Tamanho do Enchimento (Pad Length)**
 - **O quê:** Um campo que indica quantos bytes de enchimento (*padding*) foram inseridos.
 - **Por quê:** Permite à entidade destinatária saber exatamente quanto enchimento deve remover após a decifração.
- **3. Próximo Cabeçalho (Next Header)**
 - **O quê:** Identifica o tipo de dados da carga útil (o datagrama original).
 - **Por quê:** Informa ao S.O. do destinatário para qual protocolo entregar o pacote decifrado (ex: TCP, UDP, ICMP).
- **Resumo do Processo de Cifragem:**
 - Os [Dados da Carga Útil (Datagrama IP original)] e o [Trailer ESP] são concatenados.
 - O resultado concatenado é então **cifrado** como uma única unidade.

O MAC de Autenticação ESP (ICV)

- Além da criptografia, a entidade remetente anexa uma **Autenticação MAC** (MAC - Message Authentication Code) ou ICV.
- **Sobre o que o MAC é calculado?**
 - O remetente calcula o MAC sobre **toda o datagrama**.
- **O que o cálculo do MAC?**
 - O Cabeçalho ESP (SPI + Seq. Number) - (em aberto).
 - O Datagrama IP original (já criptografado).
 - O Trailer ESP (Padding, etc.) (já criptografado).
- **Como o MAC é calculado? (Revisão)**
 - O remetente anexa uma **chave MAC secreta** definida na SA.
 - Em seguida, calcula um **hash de tamanho fixo** do resultado (ex: HMAC-MD5 ou HMAC-SHA1).
 - Este hash é o MAC/ICV, que é anexado ao final do pacote.

Processamento no Destino (Exemplo: Roteador R2)

- Quando R2 recebe o datagrama IPsec (destino: R2, protocolo: 50):
- **1. Identificação da SA:**
 - R2 entende que é um pacote ESP (Protocolo 50).
 - Ele lê o campo **SPI** (no cabeçalho ESP) para determinar a qual SA o datagrama pertence (procurando no SAD).
- **2. Verificação de Autenticidade/Integridade:**
 - R2 calcula o MAC sobre o datagrama (usando a chave da SA).
 - Compara o MAC calculado com o valor no campo MAC ESP.
 - Se forem compatíveis: R2 sabe que (A) o pacote veio de R1 e (B) não foi adulterado.
- **3. Verificação Antireplay:**
 - R2 verifica o campo **Número de Sequência** para garantir que o datagrama é novo (e não um pacote repetido).

- **4. Decodificação (Descriptografia):**
 - R2 decodifica a unidade cifrada (payload + trailer) usando o algoritmo e a chave de criptografia da SA.
- **5. Extração do Pacote Original:**
 - R2 remove o enchimento (padding) e extrai o datagrama IP original (que estava cifrado).
- **6. Encaminhamento:**
 - R2 finalmente encaminha o datagrama IP original (agora em claro) para a rede da filial, em direção ao seu destino final (ex: 172.16.2.48).

- **O Desafio: Como criar as Associações de Segurança (SAs)?**
- **Opção 1: "Teclagem Manual" (Manual Keying)**
 - **Cenário:** VPNs com um número pequeno de pontos finais (ex: 2 roteadores).
 - **Ação:** O administrador de rede pode inserir *manualmente* as informações da SA (algoritmos, chaves, SPIs) nos SADs das entidades.
- **O Problema desta Abordagem:**
 - É claramente **impraticável** para uma VPN grande.
 - (Ex: Redes com centenas ou milhares de roteadores e hospedeiros IPsec).
- **Opção 2: O Mecanismo Automático (IKE)**
 - Implementações grandes e geograficamente distribuídas exigem um mecanismo automático para a criação das SAs.
 - O IPsec utiliza para isso o protocolo **IKE (Internet Key Exchange)**.
 - Especificado na **RFC 5996**.

IKE: Funções Principais do Protocolo

- O protocolo IKE (Internet Key Exchange) é o mecanismo automático para criar SAs.
- **Responsabilidades do IKE:**
 - **1. Autenticação das Entidades**
 - Troca certificados para provar a identidade das partes (ex: R1 e R2).
 - **2. Negociação de Parâmetros**
 - Negocia os algoritmos de criptografia (ex: AES, 3DES).
 - Negocia os algoritmos de autenticação (ex: HMAC-SHA1).
 - **3. Geração de Chaves**
 - Troca informações de chave com segurança (usando Diffie-Hellman).
 - Cria as chaves de sessão (secretas) que serão usadas nas SAs do IPsec.

IKE: A Estrutura de Duas Fases

- O IKE opera em duas fases principais (contexto: R1 e R2).
- **Fase 1: O Canal Seguro**
 - Objetivo: Criar um canal seguro e autenticado **para o próprio IKE**.
 - Consiste em duas trocas de pares de mensagens.
 - Resultado: Criação de uma **IKE SA** (bidirecional).
- **Atenção: IKE SA $\neq$ IPsec SA**
 - A **IKE SA** (Fase 1) é um canal seguro para proteger as negociações do IKE.
 - As **IPsec SAs** (Fase 2) são as conexões unidirecionais usadas para proteger os dados do usuário (o tráfego da VPN).

- **Primeira Troca de Mensagens (Anônima)**

- Os dois lados (R1, R2) usam **Diffie-Hellman**.
- **Objetivo:** Criar a IKE SA bidirecional.
- Chaves são estabelecidas para criptografia e autenticação **da IKE SA**.
- Um **segredo mestre** é estabelecido (para ser usado na Fase 2).
- *Importante:* Nesta etapa, nem R1 nem R2 revelam sua identidade (não assinam nada com chaves privadas).

- **Segunda Troca de Mensagens (Autenticada)**

- Ambos os lados **revelam suas identidades** (ex: certificados) assinando as mensagens.
- *Segurança:* As identidades não são expostas a "analísadores passivos", pois esta troca já ocorre **dentro do canal seguro IKE SA** criado na primeira troca.
- *Negociação:* Os dois lados negociam os algoritmos de autenticação e criptografia que serão usados pelas futuras **SAs IPsec** (da Fase 2).

- **IKE Fase 2: Criação das SAs IPsec**

- Ao fim da Fase 1, existe um canal IKE SA seguro e autenticado.
- Na Fase 2, os dois lados usam este canal para criar as **SAs do IPsec**.
- Criam uma SA IPsec em cada direção (duas SAs unidirecionais).
- As chaves de sessão de criptografia e autenticação são estabelecidas para ambas as SAs IPsec.
- Agora, podem usar essas SAs IPsec para enviar dados seguros.

- **A Motivação para as Duas Fases**

- **Custo Computacional.**
- A Fase 1 é "cara" (envolve criptografia de chave pública: Diffie-Hellman e assinaturas RSA).
- A Fase 2 é "barata" (não envolve chave pública; usa o segredo mestre da Fase 1).
- **Vantagem:** Permite criar um *grande número* de SAs IPsec (Fase 2) com um custo computacional relativamente pequeno, após estabelecer apenas uma IKE SA (Fase 1).

Aula 13 - TLS

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
October 24, 2025

Protegendo conexões TCP: TLS

- **Contextualização:**
 - Vimos segurança na camada de rede.
 - Agora, subimos uma camada na pilha de protocolo.
- **O Objetivo do TLS:**
 - Examinar como a criptografia pode aprimorar o **TCP**.
 - (Transport Layer Security).
- **Serviços de Segurança Adicionados ao TCP:**
 - Sigilo (Confidencialidade).
 - Integridade dos Dados.
 - Autenticação do Ponto Final (End-point Authentication).
- **Padronização e Histórico:**
 - O TLS é padronizado pelo IETF na **RFC 4346**.
 - Uma versão anterior e semelhante deste protocolo é o **SSL (Secure Sockets Layer)** versão 3.

TLS: Histórico e Adoção

- **Origem:** O SSL (Secure Sockets Layer) foi originalmente projetado pela Netscape.
 - (Ideias básicas sobre proteção do TCP antecedem este trabalho, ex: Woo, 1994).
- **Sucessor:** O TLS (Transport Layer Security) é o sucessor padronizado do SSL.
- **Adoção Massiva:**
 - Suportado por todos os navegadores Web e servidores Web populares.
 - Usado pelo Gmail e por **basicamente todos os sites de comércio eletrônico** (Amazon, eBay, TaoBao).
 - Centenas de bilhões de dólares são gastos anualmente usando TLS.
- **Como o usuário identifica o TLS?**
 - Se você já comprou algo online com cartão de crédito, sua comunicação foi quase certamente via TLS.
 - A URL no navegador se inicia com **https://** (em vez de http://).

A Necessidade do TLS: Cenário de E-commerce

- **O Cenário Típico:**

- **Bob** (cliente) está navegando na Web.
- Ele acessa o site da **Alice Incorporated** (vendedor) para comprar perfume.

- **A Ação:**

- O site exibe um formulário.
- Bob insere informações sensíveis:
 - Tipo de perfume e quantidade.
 - Seu endereço.
 - O **número de seu cartão de crédito**.
- Bob clica em "Enviar".

- **A Expectativa (sem segurança):**

- Bob espera receber o perfume e a cobrança correta.
- **O Risco:** Se nenhuma medida de segurança for tomada, Bob pode ter "surpresas".

Riscos: O Cenário de E-commerce sem TLS

- **1. Se nenhuma Confidencialidade (Sigilo) for utilizada:**
 - Um invasor (Trudy) poderia **interceptar** o pedido.
 - Trudy rouba as informações do cartão de crédito de Bob.
 - Consequência: O invasor pode fazer compras à custa de Bob.
- **2. Se nenhuma Integridade dos Dados for utilizada:**
 - Um invasor poderia **modificar** o pedido de Bob em trânsito.
 - Ex: Fazer Bob comprar 10x mais frascos de perfume do que o desejado.
- **3. Se nenhuma Autenticação do Servidor for utilizada:**
 - Um servidor (mantido por Trudy) poderia **se passar** pela Alice Inc., usando o mesmo logotipo.
 - Bob insere seus dados no site falso de Trudy.
 - Consequência: Trudy fica com o dinheiro e some, ou pratica **roubo de identidade** (nome, endereço, cartão).

TLS: Além do HTTP e Posição na Pilha

- **Uso Comum: HTTPS**

- Muitas vezes, o TLS é usado para oferecer segurança em transações que ocorrem pelo HTTP.

- **O Verdadeiro Alcance: Qualquer Aplicação TCP**

- Como o TLS protege o **TCP**, ele pode ser empregado por **qualquer aplicação** que execute sobre o TCP (ex: FTP, SMTP, etc.).

- **Interface para o Desenvolvedor (API)**

- O TLS provê uma Interface de Programação de Aplicações (API) com Sockets, muito semelhante à API de Sockets do TCP.
- Para usar o TLS, a aplicação simplesmente inclui as classes ou bibliotecas SSL/TLS.

- **Localização na Pilha de Protocolos**

- Tecnicamente, o TLS reside na **Camada de Aplicação**.
- Do ponto de vista do **desenvolvedor**, ele é visto como um **protocolo de transporte** que provê serviços do TCP aprimorados com serviços de segurança.
- (Conforme Figura 8.24: TLS fica entre a Aplicação e o TCP).

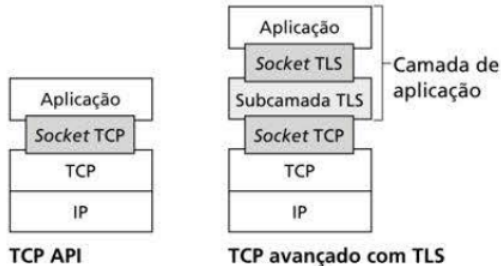


Figura 8.24 Embora o TLS resida tecnicamente na camada de aplicação, do ponto de vista do desenvolvedor, ele é um protocolo da camada transporte.

Arquitetura do TLS

- O TLS é projetado para **usar o TCP** para prover um serviço seguro, confiável (reliable) e fim-a-fim.
- O TLS não é um protocolo único, mas sim **duas camadas** de protocolos.
- **Camada 1: O Protocolo de Registro (Record Protocol)**
 - Fica "em cima" do TCP.
 - Fornece os serviços básicos de segurança (criptografia, integridade) para os protocolos da camada superior.
- **Camada 2: Protocolos Superiores (Gerenciamento TLS)**
 - São protocolos **específicos do TLS**, usados na **gestão** das trocas TLS.
 - São três protocolos que rodam "em cima" do Record Protocol:
 - Protocolo Handshake (Handshake Protocol)
 - Protocolo de Troca de Especificação de Cifra (Change Cipher Spec Protocol)
 - Protocolo de Alerta (Alert Protocol)
- **Camada de Aplicação (Exemplo)**
 - O **HTTP** (Hypertext Transfer Protocol), usado para interação Web cliente/servidor, pode operar "em cima" do TLS.

A pilha SSL/TLS

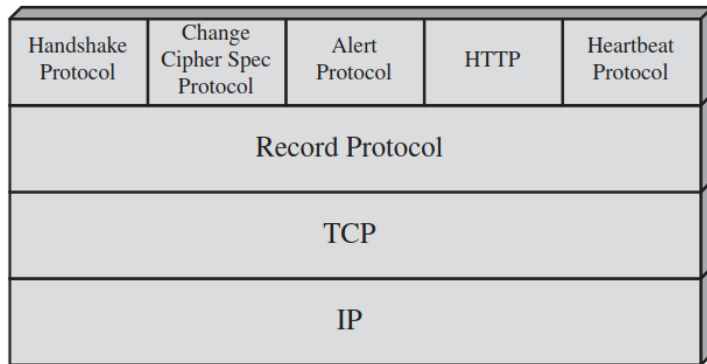


Figure 22.4 SSL/TLS Protocol Stack

Conceitos Importantes: Conexão TLS vs. Sessão TLS

- Existem dois conceitos centrais: Conexão e Sessão.
- **Conexão (Connection)**
 - É um transporte (no sentido OSI) que provê um serviço.
 - No TLS, é uma relação *peer-to-peer*.
 - As conexões são **transitórias** (passageiras).
 - **Toda conexão está associada a UMA sessão.**
- **Sessão (Session)**
 - É uma associação entre um cliente e um servidor.
 - Criada pelo **Handshake Protocol**.
 - Define um conjunto de **parâmetros de segurança criptográficos** (chaves, algoritmos, etc.).
 - Estes parâmetros podem ser **compartilhados** entre múltiplas conexões.
 - **Objetivo:** Evitar a negociação (cara) de novos parâmetros de segurança para cada nova conexão.
- **Relacionamento:**
 - Pode haver **múltiplas conexões seguras** entre um cliente e um servidor.
 - Na prática, essas múltiplas conexões compartilham (reutilizam) os parâmetros de uma **única sessão**.

O protocolo de registro do TLS

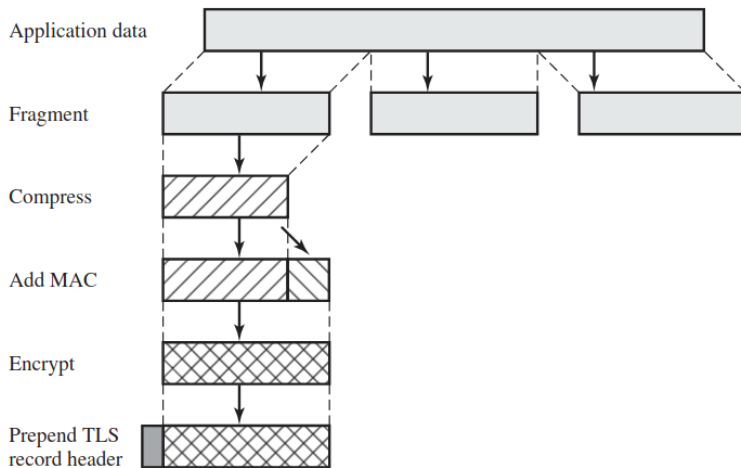


Figure 22.5 TLS Record Protocol Operation

Protocolo de Registro TLS (Record Protocol): Serviços

- O Protocolo de Registro (SSL/TLS Record Protocol) provê dois serviços básicos para as conexões TLS:
- **1. Confidencialidade (Sigilo)**
 - O Protocolo Handshake (visto mais tarde) define uma **chave secreta compartilhada**.
 - Esta chave é usada para **criptografia simétrica** dos payloads (dados) do TLS.
- **2. Integridade da Mensagem**
 - O Protocolo Handshake também define (outra) **chave secreta compartilhada**.
 - Esta chave é usada para formar um **MAC (Message Authentication Code)**.
- **Tipos de Conteúdo (Content Types) transportados:**
 - *change_cipher_spec* (Protocolo de Gerenciamento TLS)
 - *alert* (Protocolo de Gerenciamento TLS)
 - *handshake* (Protocolo de Gerenciamento TLS)
 - *application_data* (Dados da Aplicação, ex: HTTP).
- **Obs:** O conteúdo de *application_data* é **opaco** para o TLS (ele não sabe o que é HTTP, apenas o protege).

Protocolo Change Cipher Spec

- Esse protocolo utiliza o Protocolo de Registro (TLS Record Protocol).
- É o **mais simples** de todos os protocolos de gerenciamento do TLS.
- **Estrutura do Protocolo:**
 - Consiste em uma **única mensagem**.
 - Essa mensagem contém apenas **um byte**.
 - O valor desse byte é **1**.
- **Finalidade (Ação Chave):**
 - A sua única finalidade é sinalizar uma transição.
 - A mensagem "Change Cipher Spec" **NÃO** é parte do Handshake, mas sim um sinal *entre* fases.
- **O Que a Mensagem Causa?**
 - Atualiza (ativa) o cipher suite a ser usado nesta conexão a partir deste ponto.

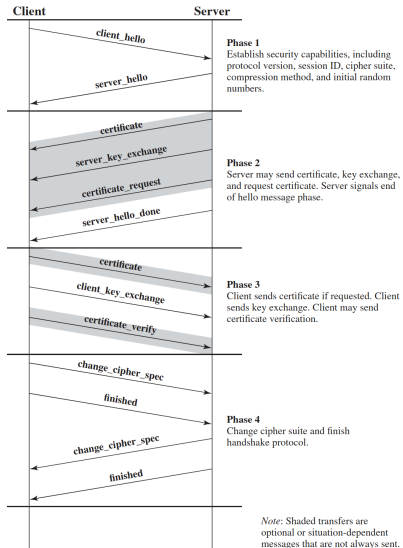
Protocolo de Alerta (Alert Protocol)

- **Finalidade:** Usado para transmitir alertas relacionados ao TLS para o outro lado.
- **Segurança:** Assim como os dados da aplicação, as mensagens de alerta são **comprimadas e criptografadas**.
- **Estrutura da Mensagem (2 bytes):**
 - **Primeiro Byte (Severidade):**
 - 'warning(1)': Um aviso (não fatal).
 - 'fatal(2)': Um erro fatal.
 - **Segundo Byte (Código):**
 - Indica o alerta específico (ex: 'bad_record_mac', 'close_notify').
- **Consequência de Alertas Fatais:**
 - Se o nível for 'fatal', o TLS **termina a conexão imediatamente**.
 - Outras conexões na mesma sessão podem continuar.
 - Nenhuma *nova conexão* pode ser estabelecida nesta sessão.
- **Exemplos de Alertas:**
 - **Exemplo Fatal:** 'incorrect MAC' (MAC incorreto).
 - **Exemplo Não Fatal (Warning):** 'close_notify' (notifica o destinatário que o remetente não enviará mais mensagens nesta conexão).

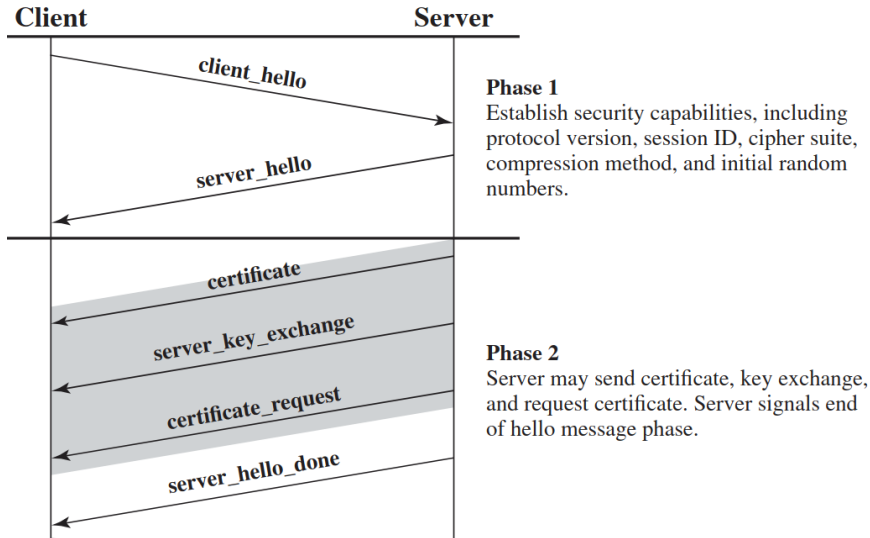
Protocolo Handshake (Handshake Protocol)

- É a parte **mais complexa** do TLS.
- Usado **antes** que quaisquer dados da aplicação sejam transmitidos.
- **Permite que o Servidor e o Cliente:**
 - Autentiquem-se mutuamente (especialmente o servidor ao cliente).
 - Negociem um algoritmo de **criptografia**.
 - Negociem um algoritmo de **MAC** (integridade).
 - Negociem as **chaves criptográficas** a serem usadas para proteger os dados.
- Consiste em uma série de mensagens trocadas entre cliente e servidor, divididas em quatro fases (conforme Figura 22.6).

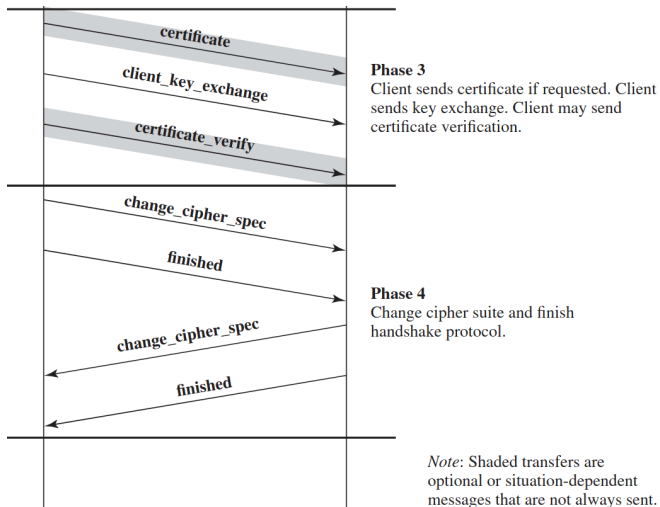
Protocolo Handshake do TLS



Protocolo Handshake do TLS (zoom 1)



Protocolo Handshake do TLS (zoom 2)



2.6 Handshake Protocol Action

As Quatro Fases do Handshake (Visão Geral)

- O protocolo consiste em uma série de mensagens trocadas.
- A troca pode ser vista como tendo quatro fases distintas:
- **Fase 1: Iniciação e Capacidades**
 - Inicia a conexão lógica (`client_hello`, `server_hello`).
 - Estabelece as capacidades de segurança (o que ambos suportam).
- **Fase 2: Autenticação do Servidor**
 - O servidor envia seu certificado e (possivelmente) informações de chave.
 - Termina com `server_done`.
- **Fase 3: Resposta do Cliente**
 - O cliente verifica o servidor e envia suas informações de chave (ex: `ClientKeyExchange`).
- **Fase 4: Finalização**
 - Completa a configuração da conexão segura, ativando os novos parâmetros (`ChangeCipherSpec`, `Finished`).

Handshake - Fase 1 (Início: Client Hello)

- A troca é iniciada pelo **cliente**, que envia uma mensagem `client_hello`.
- **Parâmetros do `client_hello`:**
- **Version:** A **versão mais alta** do TLS que o cliente entende.
- **Random:** Uma estrutura aleatória gerada pelo cliente.
 - Composição: 32-bit timestamp + 28 bytes aleatórios (de um gerador seguro).
 - Usado para prevenir ataques de *replay*.
- **Session ID:**
 - **Valor diferente de zero:** Cliente deseja atualizar parâmetros de uma conexão existente ou criar nova conexão nesta sessão (retomada de sessão).
 - **Valor zero:** Cliente deseja estabelecer uma nova conexão em uma nova sessão.
- **CipherSuite:**
 - Uma **lista** das combinações de algoritmos criptográficos que o cliente suporta.
 - Em **ordem decrescente de preferência**.
 - Cada item define: algoritmo de troca de chave E um CipherSpec (algoritmo de cifra/MAC).
- **Compression method:** Lista de métodos de compressão suportados.

Handshake - Fase 1 (Resposta: Server Hello)

- Após enviar o `client_hello`, o cliente aguarda a resposta.
- O servidor responde com uma mensagem `server_hello`.
- **Conteúdo do `server_hello`:**
 - Contém os **mesmos parâmetros** do `client_hello`.
 - **Porém:** O servidor agora **selecionou UMA opção** de cada lista (as que serão usadas).
 - **Version:** A versão que será usada (escolhida pelo servidor).
 - **Random:** O valor aleatório do **servidor**.
 - **Session ID:** O ID da sessão (se for retomada ou nova).
 - **CipherSuite:** A **única** suíte de cifras (escolhida pelo servidor) que será usada.
 - **Compression method:** O **único** método (escolhido pelo servidor) que será usado.

Handshake - Fase 2 (Autenticação do Servidor)

- Os detalhes desta fase dependem do esquema de criptografia de chave pública (definido no CipherSuite).
- **O que o servidor envia (após o `server_hello`):**
 - **1. Certificado (Certificate):**
 - (Quase sempre) O servidor envia seu certificado digital para o cliente se autenticar.
 - **2. Informação de Chave (ServerKeyExchange):**
 - (Opcional) Pode enviar informações adicionais de chave (ex: parâmetros Diffie-Hellman).
 - **3. Solicitação de Certificado (CertificateRequest):**
 - (Opcional) O servidor pode solicitar um certificado do cliente (para autenticação mútua).
 - **4. Fim da Mensagem (Server Done):**
 - **Sempre obrigatória.**
 - O servidor envia a mensagem `server_done`.
 - Indica o fim do "olá" do servidor e mensagens associadas.
 - Após enviar, o servidor aguarda a resposta do cliente.

Handshake - Fase 3 (Resposta do Cliente)

- Após receber o `server_done` do servidor:
- **1. Verificação (Ações do Cliente):**
 - O cliente deve verificar se o servidor forneceu um **certificado válido** (se foi exigido).
 - O cliente deve checar se os parâmetros do `server_hello` (CipherSuite, etc.) são aceitáveis.
- **2. Envio de Mensagens (Ações do Cliente):**
 - Se tudo estiver satisfatório, o cliente envia uma ou mais mensagens ao servidor.
 - (Ex: `ClientKeyExchange` - contendo o "pre-master secret" criptografado com a chave pública do servidor).
 - (Ex: `CertificateVerify` - se o servidor solicitou um certificado do cliente e o cliente está respondendo).

Handshake - Fase 4 (Finalização - Ativação)

- **1. Ação do Cliente:**

- Envia a mensagem `change_cipher_spec`.
- (Esta mensagem usa o *Change Cipher Spec Protocol*, não o Handshake Protocol).
- Ação: Copia o CipherSpec *pendente* (negociado) para o CipherSpec *atual* (em uso).
- Imediatamente, envia a mensagem `finished`.
- **Importante:** A mensagem `finished` já é enviada usando os **novos** algoritmos, chaves e segredos.
- A `finished` verifica se a troca de chaves e a autenticação foram bem-sucedidas.

- **2. Ação do Servidor (em resposta):**

- Envia sua própria mensagem `change_cipher_spec`.
- Transfere o CipherSpec *pendente* para o atual.
- Envia sua própria mensagem `finished` (também já criptografada).

- **3. Conclusão:**

- Neste ponto, o handshake está completo.
- Cliente e servidor podem começar a trocar dados da camada de aplicação (HTTP) de forma segura.

Protocolo Heartbeat: Definição e Contexto

- **O que é um "Heartbeat"?** (Contexto Geral)
 - Um sinal periódico (hardware ou software) usado para indicar operação normal ou para sincronizar partes de um sistema.
- **O que é um "Protocolo Heartbeat"?**
 - Usado para monitorar a **disponibilidade** de uma entidade de protocolo (verificar se o outro lado está "vivo").
- **No contexto SSL/TLS:**
 - Foi definido em 2012 na **RFC 6250**.
 - Nome oficial: "Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension".

Protocolo Heartbeat: Propósitos

- O heartbeat serve a dois propósitos principais:
- **Propósito 1: "Keep-Alive" (Verificação de Atividade)**
 - Assegura ao remetente que o destinatário ainda está "vivo".
 - É útil mesmo que não tenha havido atividade (envio de dados da aplicação) sobre a conexão TCP subjacente por um tempo.
- **Propósito 2: "Firewall Traversal" (Evitar Ociosidade)**
 - Gera atividade (tráfego) na conexão durante períodos ociosos (idle).
 - **Objetivo:** Evitar o fechamento da conexão por *firewalls* que não toleram conexões ociosas.

Protocolo Heartbeat: Operação e Negociação

- **Posição na Pilha:**
 - Roda "em cima" (on top) do Protocolo de Registro TLS (TLS Record Protocol).
- **Tipos de Mensagem (2):**
 - 'heartbeat_request'
 - 'heartbeat_response'
- **Negociação (Como é ativado?):**
 - O uso do Heartbeat é estabelecido durante a **Fase 1 do Protocolo Handshake**.
 - Cada lado (peer) indica se suporta heartbeats.
 - Se suportado, o lado indica seu modo de operação:
Modo 1 Disposto a *receber requests* e *responder* com responses.
Modo 2 Disposto *apenas a enviar* requests.

Protocolo Heartbeat: Estrutura da Mensagem

- **Fluxo:**
 - Uma 'heartbeat_request' pode ser enviada a qualquer momento.
 - Ao receber um *request*, o receptor deve responder *prontamente* com um 'heartbeat_response'.
- **Conteúdo da Mensagem request:**
 - **Payload (Carga Útil):**
 - Um conteúdo **aleatório**.
 - Tamanho: entre 16 bytes e 64 Kbytes.
 - **Payload Length (Comprimento):** O tamanho do payload.
 - **Padding (Preenchimento):** Um conteúdo aleatório adicional.
- **Requisito da Mensagem response:**
 - Deve incluir uma **cópia exata** do *payload* recebido no *request*.
- **Uso do Padding:**
 - Permite a descoberta de Path MTU (PMTU).
 - (Enviando *requests* com *padding* crescente até que a resposta falhe).

Ataques SSL/TLS: Introdução e Evolução

- Desde a introdução do SSL (1994) e a padronização do TLS, inúmeros ataques foram criados.
- **Ciclo de Ameaça e Resposta:**
 - A aparição de cada ataque necessitou mudanças (contramedidas) no protocolo, nas ferramentas de criptografia ou nas implementações.
- **Categorias Gerais de Ataque:**
 - 1. Ataques ao Protocolo Handshake.
 - 2. Ataques aos protocolos de Registro (Record) e Dados da Aplicação.
 - 3. Ataques à PKI (Infraestrutura de Chave Pública).
 - 4. Outros ataques (ex: DoS).
- **A Realidade dos Protocolos de Internet:**
 - Um protocolo "perfeito" ou implementação "perfeita" nunca é alcançado.
 - A evolução dos protocolos é determinada por um constante "vai-e-vem" (back-and-forth) entre ameaças e contramedidas.

Categoria 1: Ataques ao Protocolo Handshake

- **Objetivo:** Comprometer o Handshake Protocol.
- **Exemplo (1998, [BLEI98]):**
 - Apresentada uma abordagem baseada na exploração da **formatação e implementação** do esquema de criptografia RSA.
- **Evolução do Ataque:**
 - À medida que as contramedidas eram implementadas, o ataque foi **refinado e ajustado** (ex: [BARD12]).
 - **Objetivo do Refinamento:** Não apenas burlar as contramedidas, mas também **acelerar** o tempo do ataque.

Categoria 2: Ataques aos Protocolos de Registro e Aplicação (BEAST)

- Diversas vulnerabilidades foram descobertas nestes protocolos, levando a *patches* (correções).
- **Exemplo 1: BEAST (2011)**
 - (Browser Exploit Against SSL/TLS) - por Thai Duong e Juliano Rizzo [GOOD11].
 - Transformou o que era considerado apenas uma vulnerabilidade **teórica** em um ataque **prático**.
 - **Tipo de Ataque:** *Chosen-Plaintext Attack* (Ataque de Texto Plano Escolhido).
 - (O atacante monta o ataque escolhendo um "palpite" (guess) para o texto plano associado a um texto cifrado conhecido).
 - **Contramedida:** Patches subsequentes foram capazes de bloquear este ataque.

Categoria 2: Ataques (Continuação - CRIME)

- **Exemplo 2: CRIME (2012)**
 - (Compression Ratio Info-leak Made Easy) - pelos mesmos autores do BEAST [GOOD12b].
 - **Como Funciona:** Explora uma vulnerabilidade quando a **compressão de dados** é usada juntamente com o TLS.
 - **Resultado:** Permite ao atacante recuperar o conteúdo de *web cookies*.
 - **Ameaça Principal:** Se usado para recuperar cookies de autenticação secretos, permite ao atacante realizar **sequestro de sessão (session hijacking)**.

Categoria 3: Ataques à PKI (Infraestrutura de Chave Pública)

- **Vulnerabilidade:** A atividade de verificar a **validade de certificados X.509** está sujeita a ataques.
- **Exemplo (2012, [GEOR12]):**
 - Demonstrou que bibliotecas SSL/TLS populares sofriam de implementações **vulneráveis de validação de certificados**.
 - Foram reveladas fraquezas no código-fonte de:
 - OpenSSL
 - GnuTLS
 - JSSE (Java)
 - ApacheHttpClient
 - cURL
 - PHP, Python
 - E aplicações construídas sobre elas.

Categoria 4: Outros Ataques (Ex: DoS)

- Refere-se a ataques que não se encaixam nas categorias anteriores [MEYE13].
- **Exemplo: Ataque de Negação de Serviço (DoS) (2011)**
 - Anunciado pelo grupo hacker "The Hackers Choice" [KUMA11].
 - **Como Funciona:** Cria uma carga de processamento pesada no servidor, **sobrecarregando-o com requisições de handshake SSL/TLS**.
 - (Usando o estabelecimento de novas conexões ou a "renegociação" (renegotiation)).
 - **Por que Funciona (Assimetria):** A maioria da computação (CPU) durante um handshake é feita pelo **servidor**, não pelo cliente.
 - **Resultado:** O servidor é forçado a computar continuamente números aleatórios e chaves, levando à exaustão de recursos (DoS).

Ataque: Heartbleed (2014)

- Uma das vulnerabilidades TLS potencialmente **mais catastróficas** já descobertas.
- **Onde:** Encontrado na popular biblioteca *open-source* **OpenSSL**.
- **O Quê:** Um bug (falha) na implementação do **Protocolo Heartbeat** (RFC 6250).
- **Ponto Crítico:**
 - **NÃO** foi uma falha de design (design flaw) na especificação do TLS ou do Heartbeat.
 - Foi um **erro de programação** (programming mistake) específico da biblioteca OpenSSL.

Heartbleed: A Natureza da Vulnerabilidade

- **Revisão do Heartbeat (Comportamento Esperado):**
 - O Cliente envia um `heartbeat_request`.
 - O *request* contém um campo **Payload Length** (L) e um **Payload** (P).
 - O Servidor deve responder com uma **cópia exata** do Payload (P).
- **A Falha de Programação (O Bug):**
 - O software OpenSSL (versão vulnerável) **NÃO CHECAVA** se o tamanho real (bytes) do *payload* recebido era igual ao valor no campo *Payload Length*.
- **O Vetor de Ataque (Exploit):**
 - Um adversário envia um `heartbeat_request` malicioso:
 - Define o **Payload Length** = 64 KB (o máximo).
 - Envia um **Payload** real de apenas 16 bytes (o mínimo).

Heartbleed: O Mecanismo do Vazamento (O Exploit)

- **1. Alocação (No Servidor):**

- O servidor (vulnerável) recebe o *request*.
- Ele lê o **Payload Length (64 KB)** e aloca um *buffer* de memória deste tamanho (64 KB).

- **2. Cópia (O Bug):**

- O servidor copia o *payload* recebido (os **16 bytes** reais) para o início deste buffer.
- **O Problema:** Os 63.9 KB restantes do buffer **não são sobrescritos** (not overwritten).
- Este espaço (quase 64 KB) contém o que quer que estivesse na memória RAM do servidor naquele momento (lixo de memória).

- **3. A Resposta (O Vazamento):**

- O servidor, obedecendo o protocolo, prepara uma *heartbeat_response*.
- Ele lê o **Payload Length (64 KB)** do *request* original e envia de volta **64 KB** de dados do buffer.
- **Resultado:** O atacante recebe seus 16 bytes + 63.9 KB de **dados da memória do servidor**.

Heartbleed: O Impacto Catastrófico

- **Uma "Falha Espetacular":**
 - Ataques repetidos resultam na exposição de grandes quantidades de memória do sistema vulnerável.
- **O que a memória (não sobrescrita) poderia conter?**
 - **Chaves Privadas** (a "joia da coroa" de um servidor SSL).
 - Informações de identificação de usuário.
 - Dados de autenticação (cookies de sessão).
 - Senhas, e outros dados sensíveis.
- **A Tempestade Perfeita:**
 - **Longo Período de Exposição:** O bug não foi descoberto por vários anos.
 - **Ataque Fácil:** O *exploit* é trivial de implementar.
 - **Sem Vestígios:** O ataque não deixa rastros (logs) de que ocorreu.
- **A Escala do Problema:**
 - OpenSSL era a implementação de TLS **mais usada** no mundo.
 - Afetou finanças, bancos, e-mail, redes sociais e governos.
 - Estimativa (na época): Mais de **dois terços (2/3)** dos servidores Web da Internet usavam OpenSSL [GOOD14].